

PLAN: 本番運用テスト 実行計画

役割: SPEC.md (WHAT) を満たすための **HOW** を具体的手順レベルで定義する。SPEC.md の TC-* を本番環境で実際に誘発・観測・記録する作業順序、コマンド、依頼テンプレ、確認方法を扱う。

1. 前提条件 (テスト開始前にすべて満たすこと)

1.1 デプロイ完了確認

本テストは **HDW_Notify** が**本番アカウントにデプロイ済み**の状態を実施する。本番アカウントへの AWS CLI アクセス権限を持った状態で以下を確認:

```
# 1. Lambda function が存在する
aws lambda get-function --function-name HDW_Notify

# 2. 非機密環境変数が deploy/config-prod.yml と一致
#   (本リポジトリの設定方針: SSM / Secrets Manager は使わず、Lambda 環境変数に直接投入する。
#   非機密は config-prod.yml → デプロイ時投入、機密は GHA Secret → デプロイ時投入)
aws lambda get-function-configuration --function-name HDW_Notify \
  --query 'Environment.Variables' --output yaml
# 期待値 (非機密):
#   ENVIRONMENT_NAME=prod / TARGET_FUNCTION_NAME=HDW_Backend_Processor_0001
#   AWS_CLOUDWATCH_LOGS_WINDOW_BEFORE_MIN=30 /
#   AWS_BEDROCK_MODEL_ID=jp.anthropic.claude-opus-4-7
# 期待値 (機密、値は確認しない・存在のみ確認):
#   DISCORD_WEBHOOK_URL がキーとして存在すること

# 3. HDW_Backend_Processor_0001 の CloudWatch Alarm が HDW_Notify を invoke
#   target にしている
aws cloudwatch describe-alarms \
  --query 'MetricAlarms[?contains(AlarmActions[], `HDW_Notify`)].AlarmName'

# 4. Bedrock Opus 4.7 access
aws bedrock list-foundation-models --region ap-northeast-1 \
  | grep claude-opus-4-7
```

1.2 prompt バージョン確認

```
# 本番にデプロイされている prompt が v1.4 系であることを git tag / commit hash で照合
git log --oneline --grep="system prompt" | head -5
# 期待: 1ca2c32 / 527677f を含む release/v1.0.x のデプロイ済みコミットが現在の本番
```

1.3 Discord 観測準備

- 通知先 Discord channel に運用担当 (t.kimura) が参加していること
- スマホ / PC 双方で通知到達がわかる状態にすること

1.4 テスト ZIP 作成環境

- ローカル PC に Python 3.12 / zip / 7-Zip いずれか
- ベース ZIP を 1 サンプル入手済 (§3.1 参照)

2. クライアント連絡体制 (協力は最小限)

本テストでは AWS リソースへの操作 (テスト ZIP の `inputFiles/` 投入、Lambda 設定変更、ログ取得) はすべて運用担当 (こちら側) が AWS CLI で実施する。クライアントの操作協力が必須となるのは **TC-A-1 (通常 cron アップロードの抑止)** のみ。それ以外の TC では **事前通知** のみ行う。

2.1 連絡窓口 (本テスト開始前に確定)

項目	値
クライアント側担当者	TBD (テスト開始前に確定する)
連絡チャネル	TBD (Slack 推奨)
TC-A-1 抑止依頼から実施までのリードタイム想定	1-3 営業日
緊急停止連絡先 (致命的事態時 §6.4)	TBD

2.2 通知 / 依頼の使い分け

操作	種別	主体
通常運用への影響予告 (テスト ZIP 投入で 1 サイクル分が乱れる)	事前通知 (§2.4)	こちら側で送付
通常 cron アップロードの抑止 (TC-A-1)	依頼 (§2.5)	クライアント側で実施
Lambda 設定変更 (TC-X-1〜X-3)	事前通知 + 自己実施 (§2.4 + §5.5)	こちら側
テスト ZIP の投入 (TC-B 系・TC-S-1)	事前通知 + 自己実施 (§2.4 + §5.2)	こちら側

2.3 こちら側で実施する操作 (クライアント依頼不要)

操作	実施手段	該当 TC
S3 <code>inputFiles/</code> へのテスト ZIP 投入	<code>aws s3 cp</code>	TC-B-1〜B-6, B-11, B-12, S-1
Lambda 設定 (Timeout/MemorySize/EphemeralStorage) の一時変	<code>aws lambda update-function-configuration</code>	TC-X-1, X-2, X-3

操作	実施手段	該当 TC
更		
CloudWatch Logs の抽出	<code>aws logs start-query</code>	全 TC 共通

2.4 事前通知テンプレ (こちら側からクライアントへ Slack で送付)

件名: [HDW_Notify 運用テスト] <TC-ID> 実施予告

お世話になっております。本日 <YYYY-MM-DD HH:MM (JST)> 頃から、HDW_Notify の本番運用テストにて以下の操作を実施します。

- テスト ID: <TC-ID>
- 実施内容 (こちら側で完結します):
 - <TC-B 系の場合: 「テスト用 ZIP を inputFiles/ に投入します。通常の処理サイクルが 1 回分乱れます」>
 - <TC-X 系の場合: 「HDW_ML の <Timeout / MemorySize / EphemeralStorage> を一時的に変更します。設定はテスト終了後すぐ復元します。所要時間: 約 <X> 分」>
- 通常運用への影響:
 - 1 回分の処理サイクルが失敗 / スキップされます
 - データ蓄積の欠損が <X> 件程度発生します
- 確認のお願い (任意):
 - 通常運用への影響に問題があれば、開始前にご返信ください
- 完了報告:
 - こちらから完了次第ご連絡します

2.5 抑止依頼テンプレ (TC-A-1 のみ、クライアント側で実施が必要)

件名: [HDW_Notify 運用テスト] TC-A-1 (アップロード抑止) 依頼

お世話になっております。HDW_Notify の本番運用テストにて、以下の通り通常の S3 アップロードの一時抑止にご協力いただけますでしょうか。

- テスト ID: TC-A-1
- お願いしたい操作:
 - 通常の S3 inputFiles/ アップロードを 1 回スキップ (4 時間分)
 - 対象船: <お任せ / 指定船> / 対象時刻: <お任せ>
 - 完了したらスレッドに「<船名> の <時刻> 回をスキップしました」とご返信ください
- 期待される動作:
 - 該当時間窓に HDW_ML が起動せず、CloudWatch Alarm が発火
 - Discord channel に「未起動」系の通知が届く

ご検討よろしく申し上げます。

3. テスト ZIP の作成

3.1 ベース ZIP の入手

- クライアントから本番の典型的な ZIP (例: 直近 1 週間の任意 1 ファイル) のコピーを 1 件もらう
- 受領後、ローカルの `tmp/test-zips/base/` に保管 (gitignore 対象)
- マスキング: ベース ZIP の中身そのものは公開しない。テストにのみ使用

3.2 テスト ZIP 生成スクリプト (`scripts/make_test_zip.py`)

ベース ZIP に対し TC-ID に応じた改造を施し、新しい ZIP として書き出す Python スクリプト。

使い方:

```
python scripts/make_test_zip.py TC-B-1 tmp/test-zips/base/<base>.zip \
-o tmp/test-zips/TC-B-1/<ship>-<timestamp>.zip
```

実装スニペット:

```
#!/usr/bin/env python3
"""scripts/make_test_zip.py - テスト ZIP 生成器。

各 TC-ID に応じた改造をベース ZIP に施し、新しい ZIP として書き出す。
HDW_ML の ZIP 内構造想定:
    db01/<year>/<ship>/定時|任意/General_<timestamp>.csv
    db02/<year>/<ship>/定時|任意/<timestamp>/<sensor>.csv
"""
from __future__ import annotations
import argparse
import io
import re
import sys
import zipfile
from pathlib import Path

DB02_GENERAL_PAT = re.compile(
    r"^(^|.*?/)db02/\d{4}/[^/]+/(定時|任意)/\d{14}/General\.csv$"
)
DB02_ANY_CSV_PAT = re.compile(
    r"^(^|.*?/)db02/\d{4}/[^/]+/(定時|任意)/\d{14}/[^/]+\\.csv$"
)

def _copy_zip(src: zipfile.ZipFile, dst: zipfile.ZipFile,
              skip: callable = lambda info: False,
              transform: callable = lambda info, data: (info, data)) ->
None:
    for info in src.infolist():
        if skip(info):
            continue
        data = src.read(info.filename)
        new_info, new_data = transform(info, data)
```

```
dst.writestr(new_info, new_data)

def tc_b1(zin: zipfile.ZipFile, zout: zipfile.ZipFile) -> None:
    """db02 配下の General CSV を削除"""
    _copy_zip(zin, zout, skip=lambda i: DB02_GENERAL_PAT.match(i.filename))

def tc_b3(zin: zipfile.ZipFile, zout: zipfile.ZipFile) -> None:
    """全エントリを tampered/ 配下に押し込んでフォルダ構成違反を作る"""
    def transform(info, data):
        new_info = zipfile.ZipInfo(filename=f"tampered/{info.filename}",
                                     date_time=info.date_time)
        new_info.compress_type = info.compress_type
        return new_info, data
    _copy_zip(zin, zout, transform=transform)

def tc_b5(zin: zipfile.ZipFile, zout: zipfile.ZipFile) -> None:
    """db02 配下の最初の sensor CSV の 2 行目 1 列目を BROKEN に置換"""
    target_done = False
    def transform(info, data):
        nonlocal target_done
        if not target_done and DB02_ANY_CSV_PAT.match(info.filename) \
            and not info.filename.endswith("General.csv"):
            lines = data.decode("cp932", errors="replace").splitlines()
            if len(lines) >= 2:
                cells = lines[1].split(",")
                if cells:
                    cells[0] = "BROKEN"
                    lines[1] = ",".join(cells)
                data = "\r\n".join(lines).encode("cp932", errors="replace")
                target_done = True
        return info, data
    _copy_zip(zin, zout, transform=transform)

def tc_b6(zin: zipfile.ZipFile, zout: zipfile.ZipFile) -> None:
    """db02 配下の最初の sensor CSV の末尾 10 行を削除して行数を不足させる"""
    target_done = False
    def transform(info, data):
        nonlocal target_done
        if not target_done and DB02_ANY_CSV_PAT.match(info.filename) \
            and not info.filename.endswith("General.csv"):
            lines = data.decode("cp932", errors="replace").splitlines()
            if len(lines) > 10:
                lines = lines[:-10]
            data = "\r\n".join(lines).encode("cp932", errors="replace")
            target_done = True
        return info, data
    _copy_zip(zin, zout, transform=transform)

def tc_b12(zin: zipfile.ZipFile, zout: zipfile.ZipFile) -> None:
```

```

"""db02/General CSV から 2 列目 (= 主機負荷率を含む列の想定位置) を削除"""
def transform(info, data):
    if DB02_GENERAL_PAT.match(info.filename):
        lines = data.decode("cp932", errors="replace").splitlines()
        new_lines = [",".join([c for i, c in enumerate(l.split(","))
                                if i != 1]) for l in lines]
        data = "\r\n".join(new_lines).encode("cp932", errors="replace")
    return info, data
_copy_zip(zin, zout, transform=transform)

# TC-B-2 / TC-B-4 / TC-S-1 は ZIP 中身は無改造で出力ファイル名のみ変更
NO_CONTENT_CHANGE = {
    "TC-B-2": "drop_hyphen", # 出力 ZIP 名: <ship><timestamp>.zip
    "TC-B-4": "rename_unknown", # 出力 ZIP 名: unknown-<timestamp>.zip
    "TC-S-1": "rename_shimakaji", # 出力 ZIP 名: shimakaji-<timestamp>.zip
}

HANDLERS = {
    "TC-B-1": tc_b1,
    "TC-B-3": tc_b3,
    "TC-B-5": tc_b5,
    "TC-B-6": tc_b6,
    "TC-B-12": tc_b12,
}

def main() -> int:
    p = argparse.ArgumentParser()
    p.add_argument("tc_id")
    p.add_argument("base_zip", type=Path)
    p.add_argument("-o", "--output", type=Path, required=True)
    args = p.parse_args()

    args.output.parent.mkdir(parents=True, exist_ok=True)

    if args.tc_id in NO_CONTENT_CHANGE:
        # 中身は無改造、コピーのみ。出力ファイル名は呼び出し側で命名する想定
        args.output.write_bytes(args.base_zip.read_bytes())
        print(f"[{args.tc_id}] copied without content change. "
              f"Ensure output filename reflects the TC convention.")
        return 0

    handler = HANDLERS.get(args.tc_id)
    if not handler:
        print(f"Unknown TC: {args.tc_id}", file=sys.stderr)
        return 1

    with zipfile.ZipFile(args.base_zip, "r") as zin, \
        zipfile.ZipFile(args.output, "w", zipfile.ZIP_DEFLATED) as zout:
        handler(zin, zout)

    print(f"[{args.tc_id}] generated: {args.output}")
    return 0

```

```
if __name__ == "__main__":  
    sys.exit(main())
```

TC ごとの呼び出し例 (ベースを `<ship>-<timestamp>.zip` とする):

```
SHIP=sakura  
TS=20260427120100  
BASE=tmp/test-zips/base/${SHIP}-${TS}.zip  
  
# TC-B-1: db02/General 削除  
python scripts/make_test_zip.py TC-B-1 $BASE \  
-o tmp/test-zips/TC-B-1/${SHIP}-${TS}.zip  
  
# TC-B-2: ファイル名から - 除去 (中身無改造)  
python scripts/make_test_zip.py TC-B-2 $BASE \  
-o tmp/test-zips/TC-B-2/${SHIP}${TS}.zip  
  
# TC-B-3: tampered/ 配下に押し込み  
python scripts/make_test_zip.py TC-B-3 $BASE \  
-o tmp/test-zips/TC-B-3/${SHIP}-${TS}.zip  
  
# TC-B-4: 未登録船名で配信  
python scripts/make_test_zip.py TC-B-4 $BASE \  
-o tmp/test-zips/TC-B-4/unknown-${TS}.zip  
  
# TC-B-5: 値破損  
python scripts/make_test_zip.py TC-B-5 $BASE \  
-o tmp/test-zips/TC-B-5/${SHIP}-${TS}.zip  
  
# TC-B-6: 行数不足  
python scripts/make_test_zip.py TC-B-6 $BASE \  
-o tmp/test-zips/TC-B-6/${SHIP}-${TS}.zip  
  
# TC-B-12: 主機負荷率列削除  
python scripts/make_test_zip.py TC-B-12 $BASE \  
-o tmp/test-zips/TC-B-12/${SHIP}-${TS}.zip  
  
# TC-S-1: shimakaji にリネーム (中身無改造)  
python scripts/make_test_zip.py TC-S-1 $BASE \  
-o tmp/test-zips/TC-S-1/shimakaji-${TS}.zip
```

3.3 テスト ZIP の保管と投入

- 生成済テスト ZIP は `tmp/test-zips/<TC-ID>/<base-name>.zip` に保管 (gitignore 対象)
- 投入はこちら側で AWS CLI から直接実施:

```
# 例: TC-B-1 のテスト ZIP を本番 inputFiles/ に投入
INPUT_BUCKET="<本番 inputFiles バケット名>"
aws s3 cp tmp/test-zips/TC-B-1/sakura-20260427120100.zip \
  "s3://${INPUT_BUCKET}/inputFiles/sakura-20260427120100.zip"
```

- バケット名は実行時に確認 (本ファイルにはハードコードしない)
- 投入直前に §2.4 事前通知テンプレを送付

4. 実施モード

Mode A: 自然発生観測 (passive)

- 期間中、自然発火する alarm を全て review 対象に
- 観測のみ、誘発操作なし
- 該当 TC: **TC-B-13** (補)、**TC-X-4** (デプロイ事故)、**TC-X-5** (SDK throttling)、その他自然発生

Mode B: 運用者主導のテスト ZIP 投入 (induced-by-us)

- こちら側 (運用担当) が AWS CLI で **inputFiles/** にテスト ZIP を投入する
- クライアントの操作は **不要**。ただし「テスト投入により通常処理サイクルが 1 回乱れる」旨の **事前通知のみ** 行う (§2.4)
- 該当 TC: **TC-B-1~B-6, B-12, S-1**

Mode C: 運用者操作 (induced-operator)

- HDW_ML Lambda の設定値を一時変更して誘発するケース。本番設定への変更を伴うため、変更前スナップショット取得とロールバック手順を必ず併用する (詳細は §5.5)
- 該当 TC: **TC-X-1 (Timeout) / TC-X-2 (OOM) / TC-X-3 (/tmp)**
- 本番影響が大きいケース (S3 設定編集 / IAM 剥がし等が必要な **TC-B-7~B-10, B-14**) は v1.0 では実施対象外。別 SPEC を起こすか staging 環境で別途検証

Mode D: クライアント協力 (cooperate-with-client)

- クライアント側でしか実施できない操作 (通常 cron アップロードの抑止) を依頼するケース
- 該当 TC: **TC-A-1** のみ

5. TC 別実施フロー

すべての Mode B 系 TC は以下の共通フローに従う:

```
[Step 1] テスト ZIP 作成 (§3.2)
↓
[Step 2] §2.4 事前通知テンプレを Slack で送付
↓
[Step 3] AWS CLI で S3 inputFiles/ にテスト ZIP を投入 (§3.3)
↓
```

```

[Step 4] CloudWatch Alarm 発火を待機 (最大 4 時間 + 評価窓 30 分)
↓
[Step 5] Discord 通知到達を確認 (スマホ + PC)
↓
[Step 6] Discord embed のスクショ取得 → tmp/reviews-raw/<case_id>.png
↓
[Step 7] CloudWatch Logs を `scripts/extract_logs.sh` で抽出 + マスキング
↓
[Step 8] `scripts/new_review.sh` で review file 雛形を生成し記述
↓
[Step 9] クライアントへ完了報告

```

5.1 TC-A-1 (上流アップロード未着)

- **モード:** Mode A 優先、Mode D (抑止依頼) は補助
- **Mode A 手順:**
 - 期間中に自然発生する no_logs alarm を待つ
 - 発火したら Step 5 以降を実施
- **Mode D 手順** (自然発生が期間内に来ない場合):
 - §2.5 抑止依頼テンプレをクライアントへ送付
 - クライアント完了報告後、4-5 時間で alarm 発火
 - Step 5 以降を実施

5.2 TC-B-1 ～ B-6, B-12 (ZIP 改造系)

- **モード:** Mode B
- **手順:**
 1. §3.2 のスクリプトでテスト ZIP を作成
 2. §2.4 事前通知テンプレを送付
 3. §3.3 の `aws s3 cp` で `inputFiles/` にテスト ZIP を投入
 4. 共通フロー Step 4-9

5.3 TC-S-1 (運用上 skip すべき船 / alarm が出ないことを確認)

- **モード:** Mode B (反証ケース)
- **手順:**
 1. §3.2 でテスト ZIP を作成 (shipname を `shimakaji` に)
 2. §2.4 事前通知テンプレを送付 (期待動作は「通知が来ないこと」と明記)
 3. §3.3 の `aws s3 cp` で投入
 4. 投入から **6 時間** Discord channel を観測
 5. 通知が来なければ成功 — review file を作成 (case_type=`success_no_alarm`)
 6. 通知が来た場合は仕様違反として review file に記録

5.4 TC-B-13 (PIA データのみ欠落)

- **モード:** Mode A
- **手順:**
 - 期間中、停泊中の船 (PIA データなし) の自然な処理サイクルを観測
 - alarm が発火 **しない** こと (status=success) を CloudWatch Logs で確認

- alarm が誤発火した場合のみ review file を作成

5.5 Mode C 共通フロー (運用者操作系)

5.5.1 スナップショット取得 (scripts/snapshot_lambda.sh)

```
#!/usr/bin/env bash
# Usage: ./scripts/snapshot_lambda.sh <TC-ID>
set -euo pipefail
TC_ID="${1:?TC-ID required, e.g. TC-X-1}"
FN="HDW_Backend_Processor_0001"
mkdir -p tmp/snapshots
OUT="tmp/snapshots/${TC_ID}-before-$(date -u +%Y%m%dT%H%M%S).json"

aws lambda get-function-configuration --function-name "$FN" \
  --query '{Timeout:Timeout, MemorySize:MemorySize,
EphemeralStorage:EphemeralStorage, Environment:Environment}' \
  > "$OUT"

# 同名の "latest" シンボリック JSON を最新に更新 (復元時に参照)
cp "$OUT" "tmp/snapshots/${TC_ID}-before.latest.json"
echo "snapshot saved: $OUT"
```

5.5.2 復元 (scripts/restore_lambda.sh)

```
#!/usr/bin/env bash
# Usage: ./scripts/restore_lambda.sh <TC-ID>
set -euo pipefail
TC_ID="${1:?TC-ID required}"
FN="HDW_Backend_Processor_0001"
SNAP="tmp/snapshots/${TC_ID}-before.latest.json"
[ -f "$SNAP" ] || { echo "snapshot not found: $SNAP" >&2; exit 1; }

TIMEOUT=$(jq -r '.Timeout' "$SNAP")
MEM=$(jq -r '.MemorySize' "$SNAP")
TMP_SIZE=$(jq -r '.EphemeralStorage.Size' "$SNAP")

aws lambda update-function-configuration --function-name "$FN" \
  --timeout "$TIMEOUT" \
  --memory-size "$MEM" \
  --ephemeral-storage "Size=${TMP_SIZE}"

# 復元検証: 復元後の値が snapshot と一致するか
sleep 5
aws lambda get-function-configuration --function-name "$FN" \
  --query '{Timeout:Timeout, MemorySize:MemorySize,
EphemeralStorage:EphemeralStorage}' \
  > tmp/snapshots/${TC_ID}-after.json
diff <(jq -S 'del(.Environment)' "$SNAP") \
  <(jq -S '.' tmp/snapshots/${TC_ID}-after.json) \
```

```
&& echo "RESTORE OK" \
|| { echo "RESTORE MISMATCH" >&2; exit 2; }
```

5.5.3 共通手順 (チェックリスト)

1. **スナップショット取得**: `./scripts/snapshot_lambda.sh <TC-ID>`
2. **クライアントへ事前通知**: 設定を一時変更する旨と復元見込み時刻を §2.1 連絡窓口へ通知 (依頼ではなく予告)
3. **設定変更**: §5.6 の TC ごとのコマンドを実行
4. **次回 cron 投入を待機**: 通常の S3 アップロードサイクルを利用
5. **alarm 発火** → **Discord 通知到達** → **スクショ** → **ログ抽出**: 共通フロー Step 5-7
6. **即時復元**: `./scripts/restore_lambda.sh <TC-ID>` を実行し `RESTORE OK` を確認
7. **review file 作成**: 共通フロー Step 8
8. **復元失敗時**: §2.1 緊急停止連絡先と即時連絡 + §6.4 致命的事態対応

5.6 TC-X-1～X-3 (Lambda ランタイム限界系) 個別誘発コマンド

`FN=HDW_Backend_Processor_0001` を共通変数として:

TC-X-1 Timeout

```
# 通常処理時間を確実に下回る値に変更
aws lambda update-function-configuration --function-name "$FN" --timeout 30
```

TC-X-2 MemorySize (OOM)

```
# polars 操作で容易に枯渇する値に変更
aws lambda update-function-configuration --function-name "$FN" --memory-size 256
```

TC-X-3 EphemeralStorage (/tmp)

```
# ZIP 展開で枯渇する値に変更
aws lambda update-function-configuration --function-name "$FN" \
  --ephemeral-storage 'Size=512'
```

注意事項:

- 同時に複数値を変更しない (どの設定が原因か区別できなくなる)
- 各 TC の実施後は必ず `./scripts/restore_lambda.sh <TC-ID>` を実行
- 復元前に次の TC へ進まない

5.7 TC-X-4 (ImportError / デプロイ事故)

- **モード:** Mode A (本番環境では能動誘発しない)
- **手順:** 期間中に本番デプロイ事故が起きた場合のみ review 対象
- **v1.1 移送:** staging 環境で能動誘発する別 SPEC を別途起草 (§8 に記載)

5.8 TC-X-5 (SDK スロットリング / 一時的 5xx)

- **モード:** Mode A (完全 passive)
- **手順:** 期間中に自然発生する AWS 側 transient 障害が alarm を引き起こした場合のみ review

6. 共通作業手順

6.1 review file 作成

- ファイルパス: `specs/2026/05/19/production-operation-test/reviews/<YYYY-MM-DD>-<case_id>.md`
- `case_id` 採番: `<YYYYMMDDHHmm>-<no_logs|lambda_failure|success_no_alarm>-<seq>`

6.1.1 雛形生成スクリプト (`scripts/new_review.sh`)

```
#!/usr/bin/env bash
# Usage: ./scripts/new_review.sh <case_id> <target_tc>
# Example: ./scripts/new_review.sh 202605191230-lambda_failure-01 TC-B-1
set -euo pipefail
CASE_ID="${1:?case_id required (e.g. 202605191230-lambda_failure-01)}"
TC="${2:?target_tc required (e.g. TC-B-1)}"

# case_id から日付 (YYYY-MM-DD) を切り出し reviews/ 下のサブパスを決める
DATE_PART="${CASE_ID:0:4}-${CASE_ID:4:2}-${CASE_ID:6:2}"
DIR="specs/2026/05/19/production-operation-test/reviews"
OUT="${DIR}/${DATE_PART}-${CASE_ID}.md"
[ -f "$OUT" ] && { echo "already exists: $OUT" >&2; exit 1; }
mkdir -p "$DIR"

# case_id から case_type を抽出
CASE_TYPE=$(echo "$CASE_ID" | awk -F'-' '{print $2}')

cat > "$OUT" <<EOF
# Review: ${CASE_ID}

## メタ
- case_id: ${CASE_ID}
- alarm_fired_at: <ISO 8601 UTC> (JST: <YYYY-MM-DD HH:MM>)
- case_type: ${CASE_TYPE}
- trigger_mode: <induced-client | natural | induced-operator>
- target_tc: ${TC}
- reviewer: <name>
- reviewed_at: $(date -u +%Y-%m-%dT%H:%M:%SZ)

## 事前期待 (induced のみ)
- 投入したテスト ZIP: <ファイル名 or N/A>
```

– 期待される推論: <SPEC \${TC} の "推論されるべき状況" を要約>

Discord embed 複製 (マスキング適用後)

<embed の title / fields / suggested_actions をテキストコピー>
(原本スクショ: tmp/reviews-raw/\${CASE_ID}.png)

CloudWatch Logs (マスキング適用後)

<scripts/extract_logs.sh の出力をマスキングして貼り付け>

目視確認結果 (SPEC \${TC} "目視確認" 観点に従う)

- [] 推論が SPEC で期待した状況を言い当てているか: Yes / No
- [] suggested_actions が SPEC で期待した方向性か: Yes / No
- [] 致命的誤誘導 (機密漏洩 / 架空サービス / 差別 / 金銭請求) なし: Yes / No
- [] スキーマ / 時刻表記が共通受理条件を満たす: Yes / No
- [] 環境起因 (TC-X 系) と コード起因 (TC-B 系) の区別がついているか: Yes / No / N/A

自由コメント

<違和感・改善メモ>

E0F

echo "created: \$OUT"

6.2 マスキング

review file にログや embed 内容を貼る前に、本番識別子を所定のダミー値へ機械的に置換する。

6.2.1 置換規約

置換対象	置換後
本番アカウント ID (12 桁数字)	000000000000
本番 UUID 形式の function_request_id	00000000-0000-0000-0000-000000000001
X-Ray trace_id (1-<8hex>-<24hex>)	1-00000000-000000000000000000000001

本番アカウント ID の実値は本ファイルにもスクリプトにもハードコードしない。実行時に環境変数 `PROD_ACCOUNT_ID` から渡す。

6.2.2 マスキングスクリプト (scripts/mask_review.py)

```
#!/usr/bin/env python3
"""scripts/mask_review.py – 本番識別子を所定ダミー値に置換する stdin/stdout フィルタ。
```

Usage:

```
PROD_ACCOUNT_ID=<12digits> python scripts/mask_review.py < raw.txt >
masked.txt
```

UUID / X-Ray trace_id はパターンマッチで全置換する。

```

####
import os
import re
import sys

PROD_ACCOUNT_ID = os.environ.get("PROD_ACCOUNT_ID")
if not PROD_ACCOUNT_ID or not re.fullmatch(r"\d{12}", PROD_ACCOUNT_ID):
    print("PROD_ACCOUNT_ID env var (12 digits) required", file=sys.stderr)
    sys.exit(1)

UUID_PAT = re.compile(
    r"\b[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\b"
)
XRAY_PAT = re.compile(r"\b1-[0-9a-fA-F]{8}-[0-9a-fA-F]{24}\b")

text = sys.stdin.read()
text = text.replace(PROD_ACCOUNT_ID, "000000000000")
text = UUID_PAT.sub("00000000-0000-0000-0000-000000000001", text)
text = XRAY_PAT.sub("1-00000000-000000000000000000000001", text)
sys.stdout.write(text)

```

6.2.3 コミット前検査スクリプト (scripts/check_masking.sh)

```

#!/usr/bin/env bash
# Usage: PROD_ACCOUNT_ID=<12digits> ./scripts/check_masking.sh
# review file 配下に本番識別子が漏れていないか確認する pre-commit 用
set -euo pipefail
: "${PROD_ACCOUNT_ID:?env var required}"

REVIEW_DIR="specs/2026/05/19/production-operation-test/reviews"
[ -d "$REVIEW_DIR" ] || { echo "no reviews directory yet, skipping"; exit 0; }

HITS=$(grep -RIIn -E \
    "(${PROD_ACCOUNT_ID})\b[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\b|\b1-[0-9a-fA-F]{8}-[0-9a-fA-F]{24}\b)" \
    "$REVIEW_DIR" || true)

# マスキング後の固定値は許容
HITS=$(echo "$HITS" | grep -vE '000000000000|00000000-0000-0000-0000-000000000001|1-00000000-000000000000000000000001' || true)

if [ -n "$HITS" ]; then
    echo "MASKING LEAK DETECTED:" >&2
    echo "$HITS" >&2
    exit 1
fi
echo "masking check OK"

```

6.3 CloudWatch Logs 抽出

6.3.1 抽出 + マスキングスクリプト (`scripts/extract_logs.sh`)

```
#!/usr/bin/env bash
# Usage:
#   PROD_ACCOUNT_ID=<12digits> ./scripts/extract_logs.sh
#   <alarm_fired_at_iso8601> <case_id>
# Example:
#   PROD_ACCOUNT_ID=123456789012 ./scripts/extract_logs.sh '2026-05-19T12:30:00Z' \
#   202605191230-lambda_failure-01
set -euo pipefail

FIRED_AT="${1:?alarm_fired_at (ISO 8601, e.g. 2026-05-19T12:30:00Z)
required}"
CASE_ID="${2:?case_id required}"
: "${PROD_ACCOUNT_ID:?env var required for masking}"

LOG_GROUP="/aws/lambda/HDW_Backend_Processor_0001"
START=$(date -u -d "$FIRED_AT - 30 minutes" +%s)000
END=$(date -u -d "$FIRED_AT + 30 minutes" +%s)000

QUERY_ID=$(aws logs start-query \
  --log-group-name "$LOG_GROUP" \
  --start-time "$START" --end-time "$END" \
  --query-string 'fields @timestamp, level, status, phase, exception_name,
message, exception | sort @timestamp asc | limit 500' \
  --query 'queryId' --output text)

echo "started query: $QUERY_ID" >&2

# ポーリング (最大 60 秒)
for _ in $(seq 1 30); do
  STATUS=$(aws logs get-query-results --query-id "$QUERY_ID" --query
'status' --output text)
  case "$STATUS" in
    Complete) break ;;
    Failed|Cancelled|Timeout)
      echo "query $STATUS" >&2; exit 1 ;;
    *) sleep 2 ;;
  esac
done

mkdir -p tmp/extracted-logs

# 生取得 (マスキング前) - ローカル参照用
RAW="tmp/extracted-logs/${CASE_ID}.raw.json"
aws logs get-query-results --query-id "$QUERY_ID" > "$RAW"

# マスキング適用 (review file への貼り付け用)
MASKED="tmp/extracted-logs/${CASE_ID}.masked.txt"
```

```
jq -r '.results[] | map("\(.field)=\(.value)") | join(" ")' "$RAW" \
| PROD_ACCOUNT_ID="$PROD_ACCOUNT_ID" python scripts/mask_review.py \
> "$MASKED"

echo "raw:    $RAW (gitignore)" >&2
echo "masked: $MASKED (paste into review file)" >&2
```

tmp/extracted-logs/ は .gitignore 対象。*.masked.txt を手動で review file の ## CloudWatch Logs セクションへ貼り付ける。

6.4 致命的事態時の対応

review で **致命的誤誘導** (機密漏洩 / 架空サービス言及 / 差別 / 金銭請求誘導) を検出した場合:

1. **即時**: 本 Lambda の Reserved Concurrency を 0 に設定

```
aws lambda put-function-concurrency \
  --function-name HDW_Notify \
  --reserved-concurrent-executions 0
```

2. **5 分以内**: クライアントへ「テスト中断」を連絡 (§2.1 緊急停止連絡先)
3. **当日中**: 該当 case を specs/.../reviews/CRITICAL-<case_id>.md として記録、本 PLAN は一時凍結
4. **後日**: prompt 改修 SPEC を新規起草 → 改修 → 再デプロイ → 本テスト再開

7. v1.0 で実施対象とする TC

優先	TC	モード	クライアント協力	実施手段
P0	TC-A-1	Mode A 主 / Mode D 補	Mode D 時のみ抑止 依頼が必要	passive 待機 or 抑止 依頼
P0	TC-B-1	Mode B	事前通知のみ	こちら側で aws s3 cp
P0	TC-S-1	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-2	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-3	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-4	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-5	Mode B	事前通知のみ	こちら側で aws s3 cp

優先	TC	モード	クライアント協力	実施手段
P1	TC-B-6	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-12	Mode B	事前通知のみ	こちら側で aws s3 cp
P1	TC-B-13	Mode A	不要	passive 観測のみ
P1	TC-X-1 (Timeout)	Mode C	事前通知のみ	こちら側で Lambda 設定変更
P1	TC-X-2 (OOM)	Mode C	事前通知のみ	こちら側で Lambda 設定変更
P1	TC-X-3 (/tmp)	Mode C	事前通知のみ	こちら側で Lambda 設定変更
P2	TC-X-4 (ImportError)	Mode A	不要	passive 観測のみ
P2	TC-X-5 (Throttling)	Mode A	不要	passive 観測のみ
P2 (v1.0 ではスキップ)	TC-B-7～B-11, B-14	—	—	別 SPEC / staging で検証

7.1 v1.0 期間中の最低実施目標

- **必須:** P0 を 1 件ずつ
- **推奨:** P1 のうち 3 件以上 (TC-B 改造系 と TC-X ランタイム系のバランスを取り、両方からそれぞれ 1 件以上含めることが望ましい)
- **任意:** P1 残り / P2 はスキップまたは自然発生待ち

7.2 実施順序の推奨

1. **Week 1:** 前提条件確認 (§1) + ベース ZIP 入手 + クライアント連絡窓口の確定 (TC-A-1 のため)
2. **Week 1-2:** TC-A-1 (Mode A で待機開始) + TC-S-1 (低リスクから着手、こちら側で投入)
3. **Week 2-3:** TC-B-1 (既知バグの本番再現) + TC-X-1～X-3 のうち低リスクなもの (Timeout / OOM 等、ロールバック容易)
4. **Week 3-4:** P1 群を可能な順序で
5. **期間末:** [reviews/INDEX.md](#) 集計 → v1.1 へのフィードバック整理

8. v1.1 への持ち越し予定項目

実施を通じて確定すべき項目 (現時点では v1.0 で持たない):

- 検証期間長と最低取得件数 (現実の自然発火頻度と TC-A-1 抑止依頼のレスポンス速度を見て計量)
- 合格判定 (スコアカード Yes 率閾値 / 致命的事案ゼロ件など)
- Bedrock 実呼びコスト上限

- クライアント連絡窓口の正式確定 (\$2.1 の TBD 解消)
- 本番では実施できない TC 群 (TC-B-7～B-11, B-14) を staging 環境で実施するための別 SPEC 起草
- TC-X-4 (ImportError) を staging 環境で能動誘発するための別 SPEC 起草